

University Resource Portal — Concurrency Quick Reference

This sample PDF is included as a real university resource served by the concurrent Linux C web server.

Request flow

Browser → TCP accept() → synchronized bounded queue → POSIX worker → resource/cache layer → HTTP response.

Why concurrency matters

When many students request resources at the same time, the server does not need to finish one request before beginning the next. Multiple worker threads consume queued connections concurrently.

Benchmark purpose

The Performance Lab sends a controlled number of concurrent /health requests and measures elapsed time, successful requests and requests per second. Repeat the same workload with different worker counts to observe throughput and system behavior.

Concept	Where it appears
TCP sockets	socket(), bind(), listen(), accept()
Concurrency	POSIX worker-thread pool
Synchronization	mutex + condition variable
Queueing	bounded producer/consumer request queue
Resource serving	file access + mmap() path
Observability	Linux /proc + event journal
Recovery	SIGUSR1 fault request + worker replacement